

Сравнение и анализ оптимизаций компиляторов

Зубенин Тимофей Сергеевич

Группа: 25.Б71-мм

Кафедра: Системного программирования

Научный руководитель: инженер-исследователь Романова Зинаида Андреевна

Номер семестра практики: 2

1. Постановка задачи

Целью работы является проверка возможностей современных компиляторов и их оптимизационных стратегий для обоснованного выбора компилятора под конкретную задачу. Результаты работы полезны инженерам, разрабатывающим высокопроизводительное или критически надёжное программное обеспечение.

2. Описание предлагаемого решения

Сравнение проведено на основе статьи Филиппа Н. Хислея «Генерация высококачественного кода для программ, написанных на Си».

Анализировался ассемблерный код, полученный после трёх компиляторов: clang (версия Ubuntu 18.1.3), gcc (13.3.0) и compcert (3.17), из одного исходного файла.

Ниже приведены характеристики тестовой среды:

- Операционная система: Ubuntu 24.04.4 LTS
- Архитектура: x86-64
- Процессор: AMD Ryzen 5 7530U с Radeon Graphics

В ходе замеров времени и объёма исполняемого файла возникло две проблемы:

Проблема 1: деление на ноль.

В оригинальном коде присутствовала проверка того, как компиляторы оптимизируют деление на ноль. При компиляции выдаётся предупреждение, но сам код с делением не удаляется, что позволяет компилятору пометить часть кода как недостижимую. Этот блок мешал замерам времени и проверке оптимизаций, поэтому он был убран с помощью флага компиляции.

Проблема 2: выход за границы массива `invector5`.

В статье проверяется, в первую очередь, удаление переменной индукции; выход за пределы массива является второстепенным, но приводит к неопределённому поведению (undefined behavior). Для чистоты экспериментов и корректных замеров массив был расширен, чтобы исключить выход за границы без влияния на проверяемую оптимизацию.

Табличное представление сравнения

Для наглядности сравнения сравниваемый ассемблерный код был помещён в таблицы. Такой подход позволяет удобно сопоставлять фрагменты кода и визуально отслеживать различия между ними.

Сначала идёт таблица, сравнивающая код, сгенерированный с флагом оптимизации `-O0`, и исходный код на Си до компиляции. Затем представлена таблица, сравнивающая код при `-O0` и код при `-O3`. Рассматриваемые оптимизации взяты из статьи Филиппа Н. Хислея.

Важные элементы в ассемблерном коде выделены цветом. Цветовое выделение помогает быстро определить, была ли проведена та или иная оптимизация, или же код остался без изменений.

Использование двух таблиц вместо одной обусловлено тем, что в одну таблицу код не помещался по ширине. Разделение на две части обеспечивает лучшую читаемость и сохраняет наглядность сравнения, не требуя горизонтальной прокрутки.

Таблицы можно найти в файле "*Ход работы.odt*" по ссылке в конце отчёта

3. Эксперименты

Для каждого компилятора были выполнены сборки с тремя режимами оптимизации:

- -O0 (без оптимизации);
- -Os (оптимизация по размеру кода);
- -O3 (агрессивная оптимизация по скорости).

Измерялось среднее время работы 100 последовательных запусков (фиксированный поток) и размер исполняемого файла. Также по методике из статьи проверялось наличие различных типов оптимизаций.

3.1. Результаты замеров времени и размера

Таблица 1: Результаты для -O0

	ccomp	gcc	clang
Время работы (сек)	0,00113	0,00114	0,00115
Размер файла (бит)	17072	16992	16992

Таблица 2: Результаты для -Os

	ccomp	gcc	clang
Время работы (сек)	0,00112	0,00115	0,00113
Размер файла (бит)	17032	16992	17048

Таблица 3: Результаты для -O3

	ccomp	gcc	clang
Время работы (сек)	0,00112	0,00112	0,00110
Размер файла (бит)	17032	16992	17048

Хотя размер результирующего файла при использовании флагов -Os (оптимизация на размер) и -O3 (агрессивная оптимизация по скорости) оказывается одинаковым для каждого из рассмотренных компиляторов, полученные исполняемые файлы не совпадают по своей внутренней структуре.

Интересно, что для компилятора clang размер исполняемого файла при оптимизации -Os неожиданно оказался больше, чем при отключении оптимизации -O0 (17048 бит против 16992). На первый взгляд это противоречит самой идее оптимизации по размеру, ведь обычно ожидается, что -Os уменьшает объём кода.

Это объясняется тем, что при оптимизации по объёму компилятор сначала применяет некоторые преобразования, направленные на увеличение скорости выполнения, и только затем пытается сократить размер. В данном конкретном случае эти преобразования, повышающие скорость, привели к нежелательному росту объёма кода, что и дало такой необычный результат для clang.

Все три компилятора демонстрируют практически одинаковое время выполнения тестовой программы. Из-за её малой вычислительной сложности (отсутствие тяжёлых циклов, ресурсоёмких операций ввода-вывода или сложных математических вычислений) различия в производительности лежат в пределах погрешности измерений. Иными словами, для столь простой задачи любой из уровней оптимизации даёт сопоставимый результат.

3.2. Поддерживаемые оптимизации

В таблице приведено, какие оптимизации из рассмотренных в статье поддерживаются каждым компилятором (знак «+» означает поддержку, «-» — отсутствие, «+-» — частичную поддержку).

Таблица 4: Поддержка оптимизаций компиляторами			
Оптимизация	clang	gcc	ccomp
Размножение копий	+	-	+
Свертка констант	+	+	+
Алгебраические упрощения	+	+	+
Удаление излишних загрузок/сохранений	+	+	+
Удаление излишних присваиваний	+	+	+
Снижение мощности	+	+	-
Удаление циклов	+	+	-
Удаление переменных индукции циклов	+	+	-
Удаление общих подвыражений	+	+	-
Вынесение инвариантного кода	-	-	-
Удаление недостижимого кода	+	+	+-
Разворачивание циклов	-	-	-
Слияние циклов	-	-	-
Удаление лишних цепочек перехода	+	+	+

Как видно из таблицы, gcc и clang поддерживают подавляющее большинство рассмотренных оптимизаций. CompCert заметно отстаёт — его применение оправдано в системах с высокими требованиями к математической корректности компиляции (например, в авионике или медицинском оборудовании), где производительность вторична.

4. Заключение

В ходе выполнения работы получены следующие результаты:

- Составлена таблица для сранения возможностей компиляторов из полученных в ходе работы данных.
- Выполнены замеры времени выполнения и размера исполняемых файлов для трёх уровней оптимизации.
- Установлено, что gcc и clang обеспечивают широкий набор оптимизаций, включая удаление циклов, снижение мощности и удаление общих подвыражений, в то время как ccompert ориентирован на формальную верификацию, а не на производительность.
- Практически идентичное время работы объясняется простотой тестовых примеров; для реальных приложений различия между компиляторами могут быть существенными.

Хочется отметить, что

- Ассемблерный код clang оказался наиболее легко читаемым: компилятор достаточно строго сохраняет порядок операций, что упрощает ручной анализ.
- Код gcc воспринимается заметно сложнее из-за большего разнообразия и необычности используемых инструкций, а также активного перемешивания арифметических операций с целыми и нецелыми переменными.
- CompCert генерирует наиболее запутанный код: он дополнительно помещает адрес переменной в регистр для каждого изменения значения, что сильно затрудняет понимание при наличии нескольких переменных.

Таким образом, для рассмотренной платформы в проектах, где предполагается регулярный просмотр ассемблерного кода, предпочтительнее использовать clang. При этом clang генерирует более читаемый и структурированный ассемблерный код, что существенно упрощает его анализ и отладку.

В свою очередь, gcc может дать большее преимущество, когда важен минимальный объём исполняемого файла. Из замеров сделанных ранее, можно заметить, что по этому показателю gcc опережает остальные рассмотренный компиляторы на файле для сравнения.

Что касается CompCert, его следует выбирать осознанно и только в ситуациях, где требуется формально верифицированный компилятор. Несмотря на гарантии корректности трансляции, по набору оптимизаций он существенно уступает остальным, поэтому его применение оправданно лишь в задачах, где надёжность кода критичнее производительности.

Материалы по данной работе модно найти по адресу: https://github.com/tzubenin/practice_2sem.